

ForgeDB – Technical Documentation (Stage 3)

Project Overview

ForgeDB is a platform designed to transform raw data from CSV files, Excel spreadsheets, and REST APIs into a structured relational database. The system automatically analyzes imported data, detects relationships, generates database schemas, and deploys them into PostgreSQL databases.

The goal of ForgeDB is to reduce the time and effort required to design databases manually by providing an automated workflow from data ingestion to database deployment.

0. User Stories and Mockups

User Stories

Must Have

US-01

As a user, I want to upload CSV and Excel files, so that I can use existing datasets to generate a database.

US-02

As a user, I want to connect an external API, so that I can import data directly from external systems.

US-03

As a user, I want ForgeDB to analyze imported data automatically, so that I can understand its structure.

US-04

As a user, I want ForgeDB to generate database tables and columns automatically, so that I do not have to design the schema manually.

US-05

As a user, I want ForgeDB to detect relationships between tables, so that the generated database structure is accurate.

US-06

As a user, I want to review and modify the generated schema, so that I can customize it before deployment.

US-07

As a user, I want ForgeDB to deploy a PostgreSQL database automatically, so that the database becomes ready for use.

Should Have**US-08**

As a user, I want ForgeDB to identify missing values and duplicate records, so that I can improve data quality.

US-09

As a user, I want to visualize relationships between tables, so that I can better understand the generated schema.

US-10

As a user, I want ForgeDB to generate SQL scripts, so that I can deploy databases manually if required.

Could Have**US-11**

As a user, I want AI-powered naming suggestions for tables and columns, so that the generated schema follows best practices.

US-12

As a user, I want to export database documentation, so that I can share it with my team.

Won't Have (MVP)**US-13**

Multi-database support (MySQL, SQL Server, Oracle).

US-14

Real-time synchronization with external APIs.

Mockups**Figure 1 – Dashboard Screen**

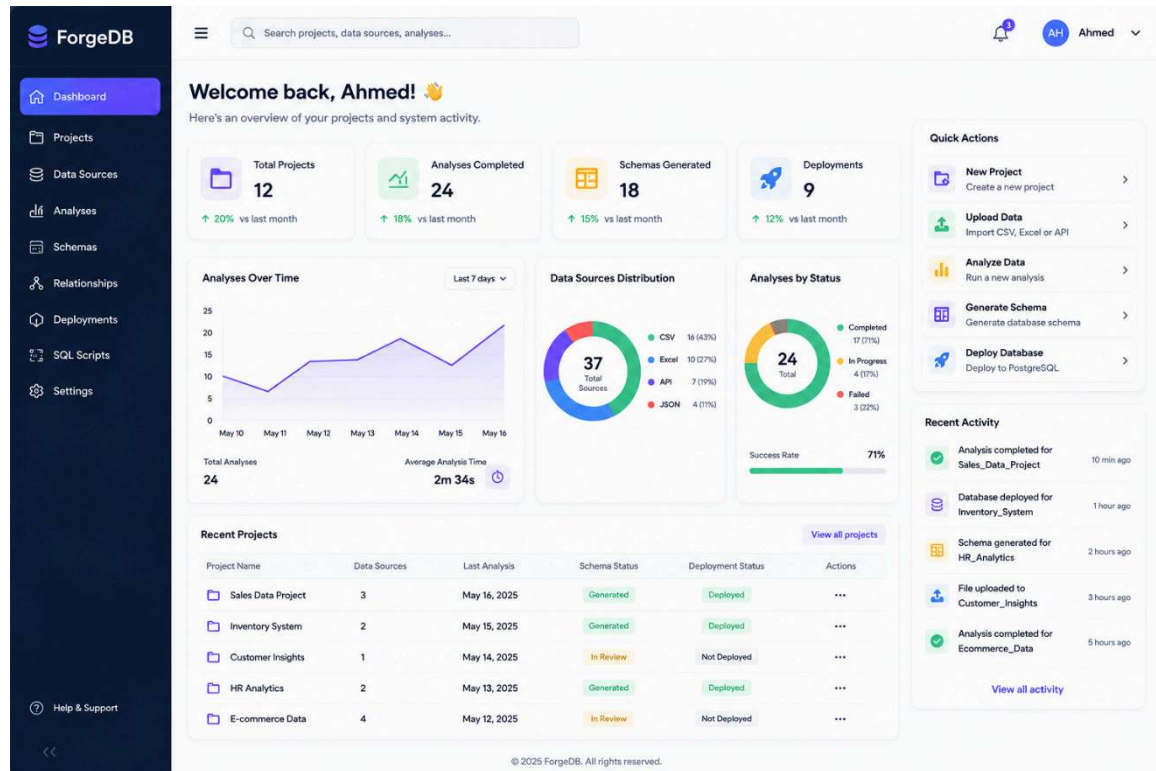


Figure 2 – Upload Data Screen

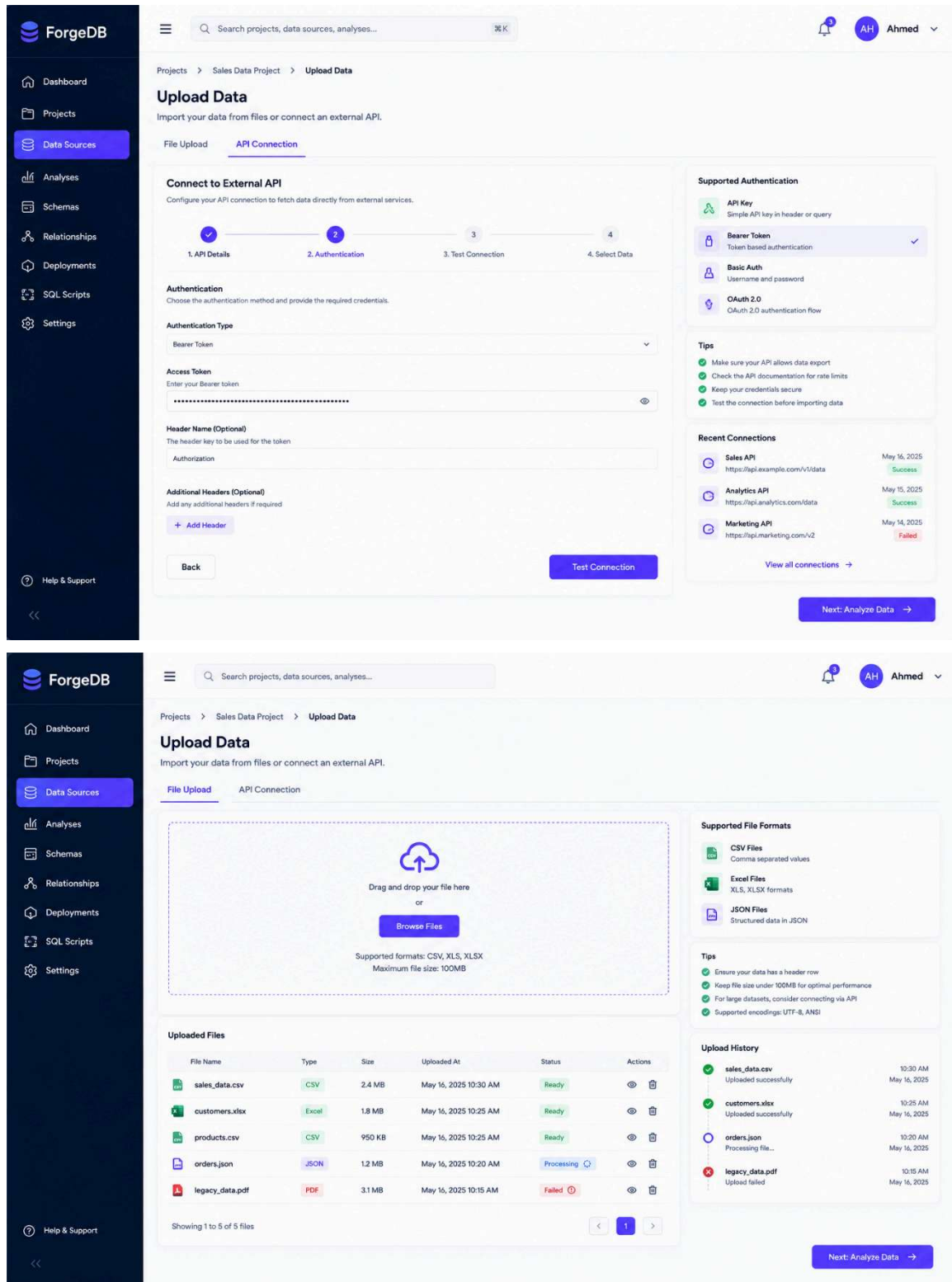


Figure 3 – Data Analysis Screen

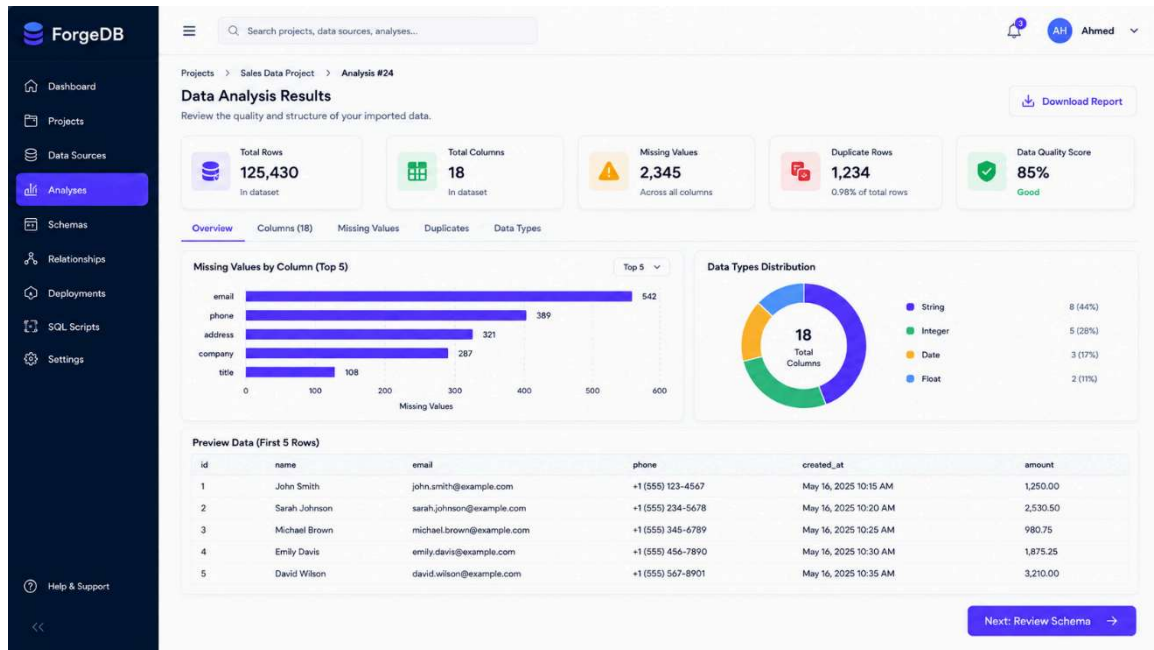
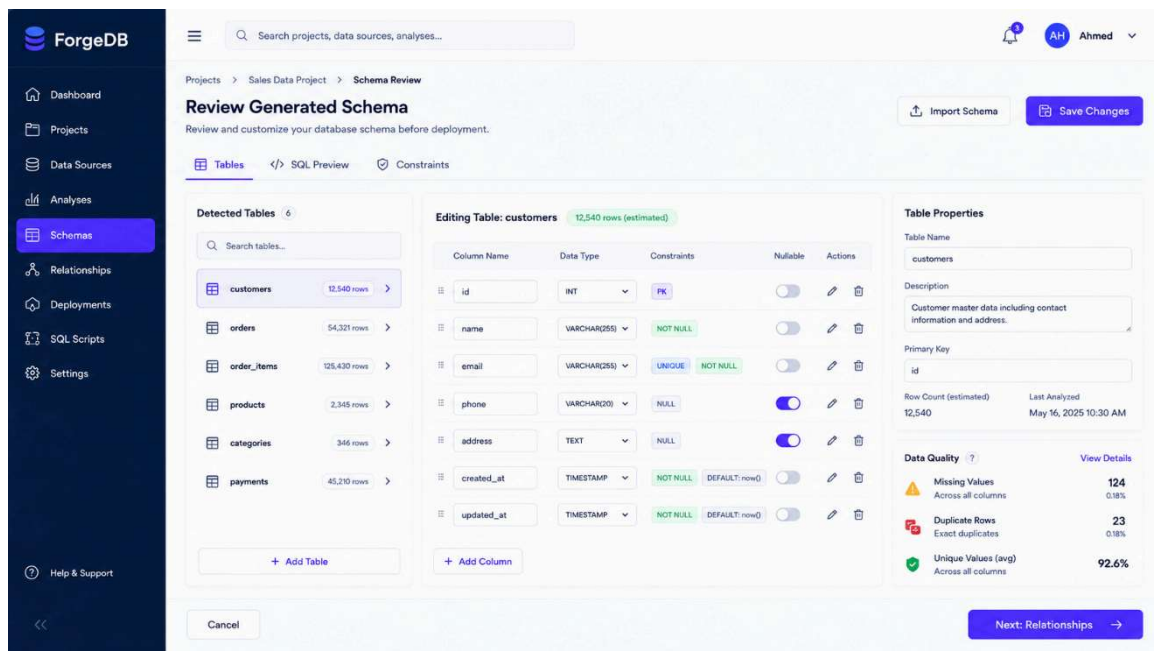


Figure 4 – Schema Review Screen



Search projects, data sources, analyses...

K

Projects

Sales Data Project

Schema Review

Import Schema

Save Changes

Tables (6)

SQL Preview

Constraints (18)

Constraints Overview

Primary Keys 6

Foreign Keys 8

Unique Constraints 3

Check Constraints 1

By Table

customers 4

orders 5

order_items 2

products 3

categories 2

payments 2

Constraints (18)

Review all constraints that will be applied to your schema.

Search constraints...

All Types

Constraint Name	Type	Table	Columns	Definition / Reference	Actions
customers_pkey	Primary Key	customers	id	PRIMARY KEY (id)	
customers_email_key	Unique	customers	email	UNIQUE (email)	
customers_phone_key	Unique	customers	phone	UNIQUE (phone)	
orders_pkey	Primary Key	orders	id	PRIMARY KEY (id)	
orders_customer_id_key	Foreign Key	orders	customer_id	REFERENCES customers(id) ON DELETE CASCADE	
orders_status_check	Check	orders	status	CHECK (status IN ('PENDING', 'PROCESSING', 'SHIPPED', 'CANCELLED'))	
order_items_pkey	Primary Key	order_items	id	PRIMARY KEY (id)	
order_items_order_id_key	Foreign Key	order_items	order_id	REFERENCES orders(id) ON DELETE CASCADE	
order_items_product_id_key	Foreign Key	order_items	product_id	REFERENCES products(id) ON DELETE RESTRICT	
products_pkey	Primary Key	products	id	PRIMARY KEY (id)	

Showing 1 to 10 of 18 constraints

<

1

2

>

Back

Cancel

Next: Relationships

Search projects, data sources, analyses...

K

Projects

Sales Data Project

Schema Review

Import Schema

Save Changes

Tables (6)

SQL Preview

Constraints (18)

SQL Preview

Review the SQL statements that will be executed to create your database schema.

Copy SQL

Download SQL

```
1 -- Table: customers
2 CREATE TABLE customers (
3   id SERIAL PRIMARY KEY,
4   name VARCHAR(100) NOT NULL,
5   email VARCHAR(100) UNIQUE,
6   phone VARCHAR(20),
7   address VARCHAR(255),
8   created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
9   updated_at TIMESTAMPTZ NOT NULL DEFAULT now()
10 );
11
12 -- Table: orders
13 CREATE TABLE orders (
14   id SERIAL PRIMARY KEY,
15   customer_id INTEGER NOT NULL REFERENCES customers(id) ON DELETE CASCADE,
16   order_date TIMESTAMPTZ NOT NULL,
17   status VARCHAR(50) NOT NULL,
18   total_amount NUMERIC(12,2) NOT NULL
19 );
20
21 -- Table: order_items
22 CREATE TABLE order_items (
23   id SERIAL PRIMARY KEY,
24   order_id INTEGER NOT NULL REFERENCES orders(id) ON DELETE CASCADE,
25   product_id INTEGER NOT NULL REFERENCES products(id),
26   quantity INTEGER NOT NULL,
27   unit_price NUMERIC(12,2) NOT NULL
28 );
29
30 -- Table: products
31 CREATE TABLE products (
32   id SERIAL PRIMARY KEY,
33   name VARCHAR(100) NOT NULL,
34   category_id INTEGER REFERENCES categories(id),
35   price NUMERIC(10,2) NOT NULL,
36   stock INTEGER NOT NULL DEFAULT 0
37 );
38
39 -- Table: categories
40 CREATE TABLE categories (
41   id SERIAL PRIMARY KEY,
42   name VARCHAR(100) NOT NULL UNIQUE,
43   description TEXT
44 );
45
46 -- Table: payments
47 CREATE TABLE payments (
48   id SERIAL PRIMARY KEY,
49   order_id INTEGER NOT NULL REFERENCES orders(id) ON DELETE CASCADE,
50   payment_date TIMESTAMPTZ NOT NULL DEFAULT now(),
51   amount NUMERIC(12,2) NOT NULL,
52   method VARCHAR(50) NOT NULL,
53   status VARCHAR(50) NOT NULL
54 );
```

Cancel

Next: Relationships

Figure 5 – Relationships Screen

The screenshot displays the ForgeDB interface for managing database relationships. The left sidebar contains navigation options: Dashboard, Projects, Data Sources, Analyses, Schemas, Relationships (selected), Deployments, SQL Scripts, and Settings. The main content area is titled "Review Relationships" and includes a search bar for projects, data sources, and analyses. Below the search bar, there's a "Review Relationships" section with a sub-header "Review and manage the relationships between your database tables." and buttons for "Auto-Detect Relationships" and "Save Changes".

The central part of the screen shows a "Relationships Overview" diagram. It lists six tables on the left: customers (4 relationships), orders (4 relationships), order_items (3 relationships), products (2 relationships), categories (1 relationship), and payments (1 relationship). The diagram itself shows a visual overview of how these tables are connected. Relationships are represented by lines between tables, with cardinalities (1, ∞) and types (Identifying Relationship, Non-Identifying Relationship) indicated. A legend at the bottom explains the symbols: 1 for One (PK), ∞ for Many (FK), a solid line for Identifying Relationship, and a dashed line for Non-Identifying Relationship.

On the right, the "Relationship Details" panel is active, showing the relationship between "orders" and "customers". It specifies a "Many-to-One" relationship, with "orders" as the "From Table" and "customers" as the "To Table". The "From Column" is "customer_id (FK)" and the "To Column" is "id (PK)". The "On Delete" and "On Update" actions are both set to "CASCADE". The "Relationship Type" is "Non-Identifying". At the bottom of this panel are buttons for "Edit Relationship" and "Delete Relationship". A "Next: Deploy Schema" button is located at the bottom right of the main content area.

Figure 6 – Deployment Screen

The screenshot displays the ForgeDB interface for deploying a database. The left sidebar is the same as in Figure 5, with "Deployments" selected. The main content area is titled "Deploy Database" and includes a sub-header "Review the generated SQL and deploy your PostgreSQL database." and buttons for "Download SQL" and "Deploy Database".

The "Deployment Configuration" section shows the "Database Type" set to "PostgreSQL", the "Database Name" as "sales_database", and a "Description (Optional)" field. A green checkmark indicates the database is "Ready to deploy" with the message "The generated SQL is valid and ready to be deployed." Below this, the "Deployment Summary" shows 6 Tables, 9 Relationships, and 18 Constraints.

The "SQL Preview" section displays the generated SQL code for creating the database schema. The code includes CREATE TABLE statements for customers, categories, products, orders, order_items, and payments, along with their respective primary keys, foreign keys, and constraints. A "Copy SQL" button is located at the top right of the SQL preview area. At the bottom of the screen, there are buttons for "Back", "Download SQL", and "Deploy Database".

1. System Architecture

Architecture Overview

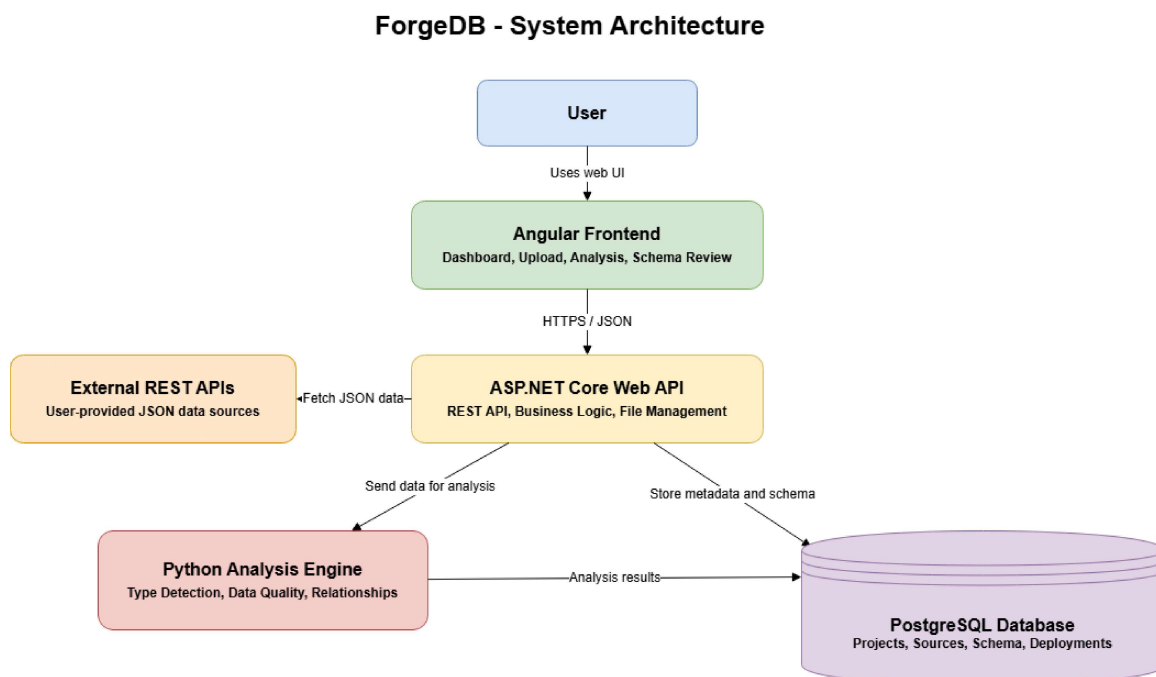
ForgeDB follows a layered architecture consisting of four main layers:

1. Angular Frontend
2. ASP.NET Core Backend
3. Python Analysis Engine
4. PostgreSQL Database

The frontend handles user interactions, the backend manages business logic and API requests, the analysis engine processes imported data and generates schema recommendations, and PostgreSQL stores project metadata and generated database structures.

Architecture Diagram

Figure 7 – ForgeDB Architecture Diagram



Data Flow

1. User uploads a file or connects an API.
 2. Angular sends the request to ASP.NET Core API.
 3. ASP.NET Core forwards the data to the Python Analysis Engine.
 4. The Analysis Engine detects data types, relationships, duplicates, and missing values.
 5. Analysis results are stored in PostgreSQL.
 6. Generated schema suggestions are returned to the user.
 7. User reviews the schema and deploys the database.
-

2. Components, Classes, and Database Design

Frontend Components

DashboardComponent

Displays project statistics, recent projects, and system overview.

UploadComponent

Handles file uploads and API connections.

DataAnalysisComponent

Displays analysis results and detected structures.

SchemaReviewComponent

Allows users to review and edit generated schemas.

RelationshipDiagramComponent

Visualizes detected relationships.

DeploymentComponent

Handles SQL generation and database deployment.

Backend Classes

User

Attributes:

- Id
- FirstName
- LastName
- Email
- PasswordHash
- Role

Methods:

- Register()
- Login()
- UpdateProfile()

Project

Attributes:

- Id
- UserId
- Name
- CreatedAt

Methods:

- CreateProject()
- GetProject()
- DeleteProject()

DataSource

Attributes:

- Id
- ProjectId
- SourceType
- SourceName

Methods:

- UploadFile()
- ConnectApi()

DataAnalysis

Attributes:

- Id
- ProjectId
- MissingValuesCount
- DuplicateRowsCount
- DetectedTablesCount

Methods:

- DetectDataTypes()
- DetectDuplicates()
- DetectMissingValues()
- GenerateAnalysisReport()

GeneratedTable

Attributes:

- Id
- ProjectId
- TableName

Methods:

- CreateTableSuggestion()
- UpdateTable()

GeneratedColumn

Attributes:

- Id
- TableId
- ColumnName
- DataType
- IsPrimaryKey

- IsForeignKey
- IsNullable

Methods:

- DetectColumnType()
- MarkAsPrimaryKey()
- MarkAsForeignKey()

Relationship

- Id
- ParentTableId
- ChildTableId
- RelationshipType

Methods:

- DetectRelationship()
- UpdateRelationship()

DatabaseDeployment

Attributes:

- Id
- ProjectId
- DatabaseName
- Status

Methods:

- GenerateSQL()
- DeployDatabase()

Database Design

Users

- id
- first_name
- last_name
- email

- password_hash
- role
- created_at

Projects

- id
- user_id
- name
- created_at

DataSources

- id
- project_id
- source_type
- source_name
- source_url

DataAnalyses

- id
- project_id
- missing_values_count
- duplicate_rows_count
- detected_tables_count

GeneratedTables

- id
- project_id
- table_name

GeneratedColumns

- id
- table_id

- column_name
- data_type
- is_primary_key
- is_foreign_key
- is_nullable

Relationships

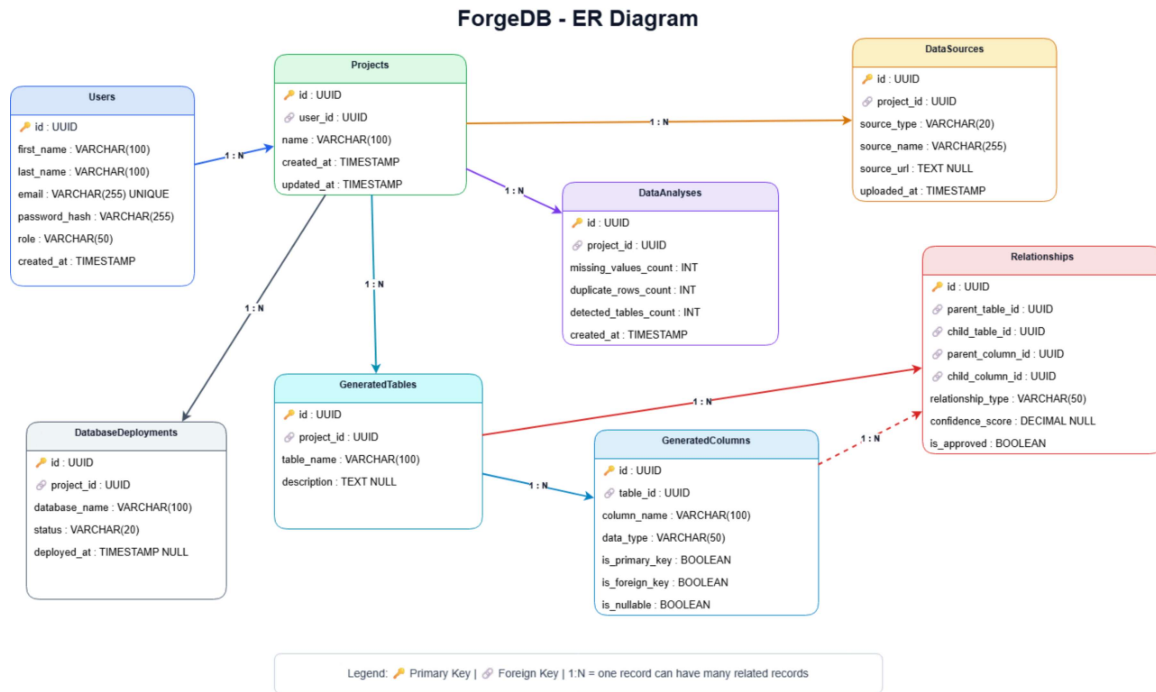
- id
- parent_table_id
- child_table_id
- relationship_type

DatabaseDeployments

- id
- project_id
- database_name
- status

ER Diagram

Figure 8 – ForgeDB ER Diagram

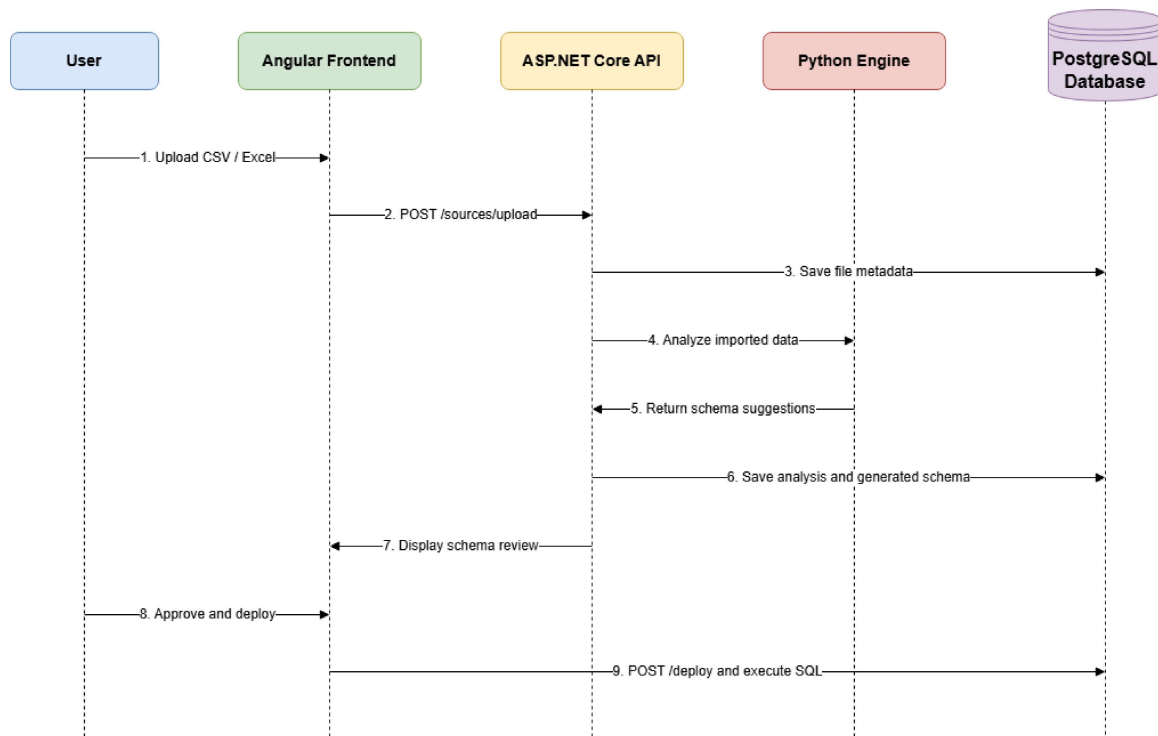


3. High-Level Sequence Diagrams

Upload and Analyze Data

Figure 9 – Upload & Analysis Sequence Diagram

ForgeDB - Upload, Analyze, and Deploy Sequence

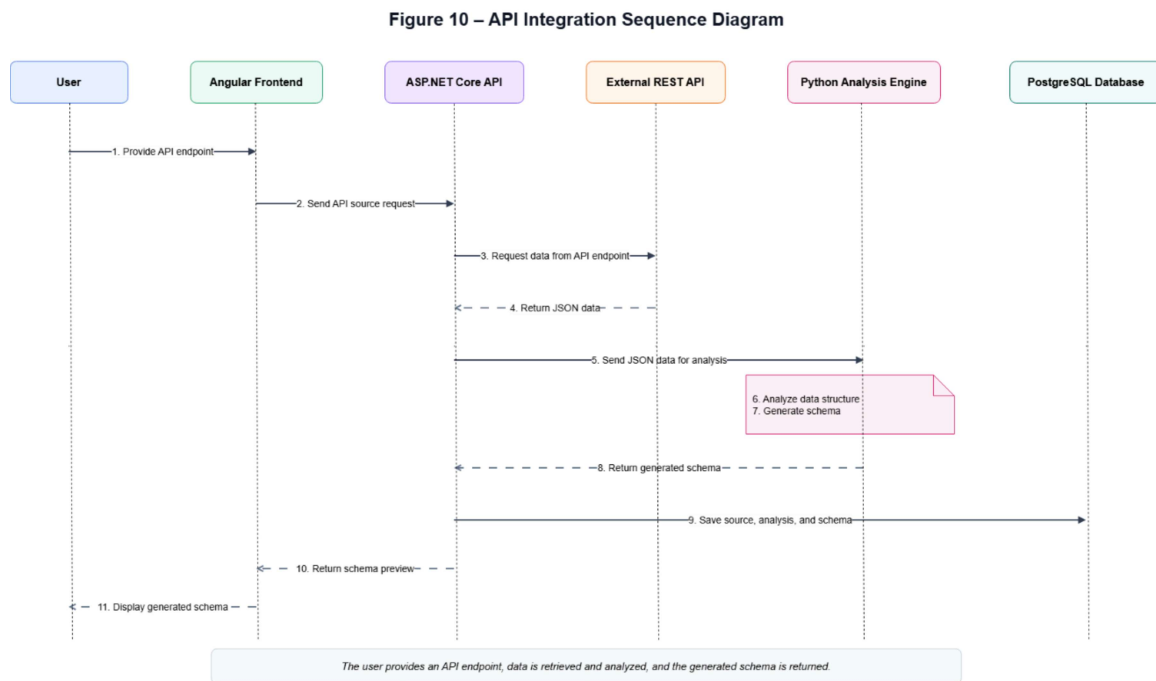


Description:

The user uploads a file, the backend processes it through the analysis engine, stores the results, and returns schema suggestions.

Connect External API

Figure 10 – API Integration Sequence Diagram



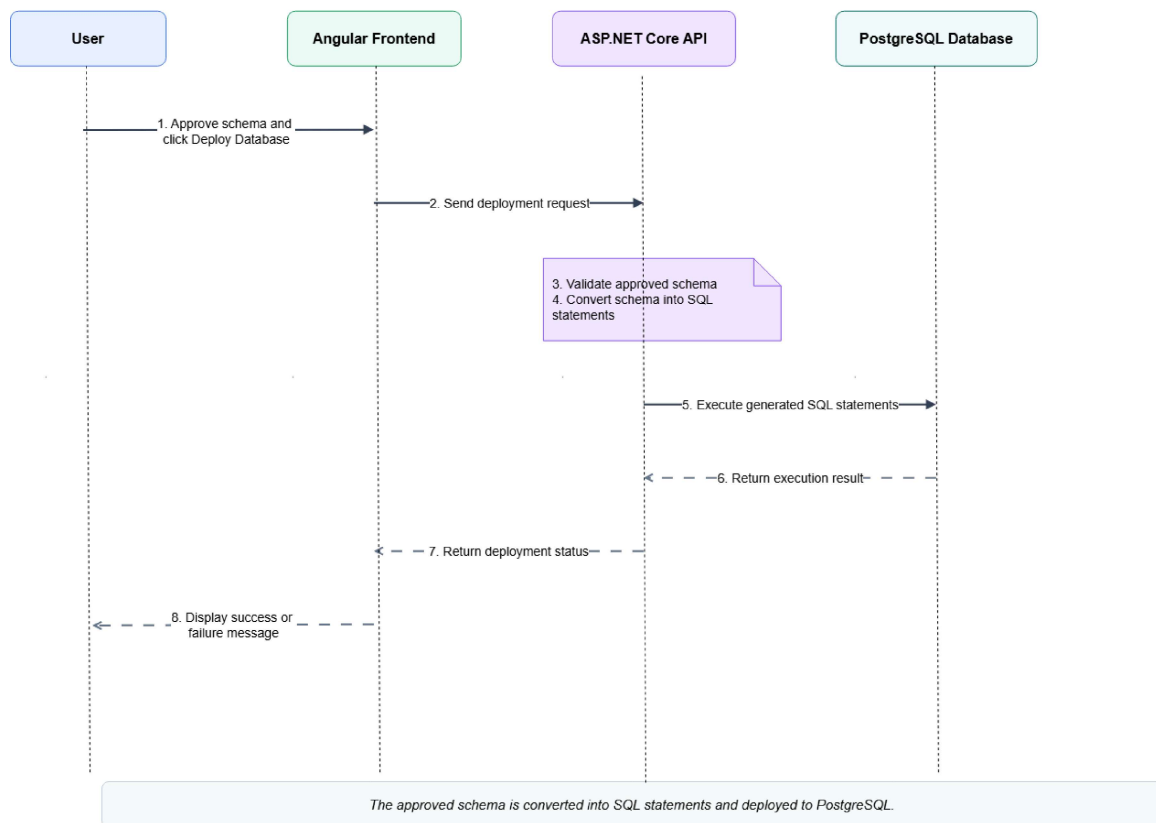
Description:

The user provides an API endpoint, data is retrieved and analyzed, and the generated schema is returned.

Deploy Database

Figure 11 – Database Deployment Sequence Diagram

Figure 11 – Database Deployment Sequence Diagram



Description:

The approved schema is converted into SQL statements and deployed to PostgreSQL.

4. External and Internal APIs

External APIs

ForgeDB supports user-provided REST APIs as external data sources. These APIs return JSON data which is analyzed and transformed into database schemas.

Internal API Endpoints

Create Project

POST /api/projects

Upload File

POST /api/projects/{projectId}/sources/upload

Connect API

POST /api/projects/{projectId}/sources/api

Analyze Data

POST /api/projects/{projectId}/sources/{sourceId}/analyze

Generate Schema

POST /api/projects/{projectId}/schema/generate

Update Schema

PUT /api/projects/{projectId}/schema

Get Relationships

GET /api/projects/{projectId}/relationships

Deploy Database

POST /api/projects/{projectId}/deploy

5. SCM and QA Strategies**Source Control Management**

The project uses Git and GitHub for version control.

Branching Strategy

- main
- develop
- feature/*

Each feature is developed in a separate branch and merged through Pull Requests after review.

Code Review

All Pull Requests must be reviewed and approved by at least one team member before merging.

Quality Assurance

Unit Testing

Tools:

- xUnit
- Jasmine
- Karma

Integration Testing

Tools:

- Postman
- Swagger

End-to-End Testing

Tool:

- Cypress

Manual Testing

Critical user workflows will be manually tested before release.

6. Technical Justifications

Angular

Angular was selected because it provides a scalable component-based architecture and strong TypeScript support.

ASP.NET Core

ASP.NET Core was chosen because of its performance, security, and support for building RESTful APIs.

Python

Python was selected for data analysis because of its extensive ecosystem for data processing and future AI integration.

PostgreSQL

PostgreSQL was selected because ForgeDB focuses on relational database generation and relationship management.

REST Architecture

REST APIs provide a simple and scalable communication model between frontend and backend components.

GitHub Flow

GitHub Flow was selected because it simplifies collaboration, code reviews, and version management.

Testing Strategy

A combination of unit testing, integration testing, end-to-end testing, and manual testing ensures system quality and reliability.

Overall Architecture

Separating the frontend, backend, analysis engine, and database improves maintainability, scalability, and future extensibility.